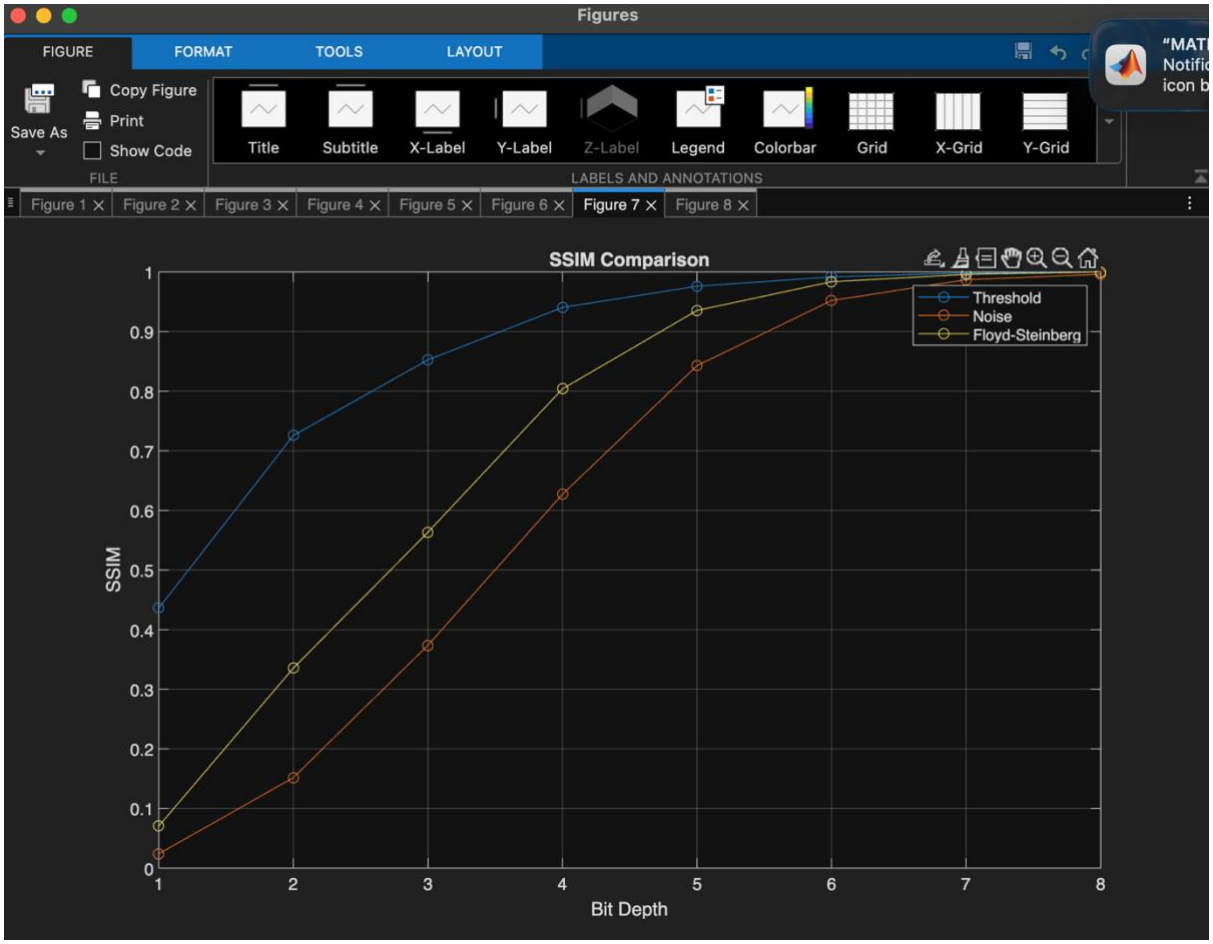
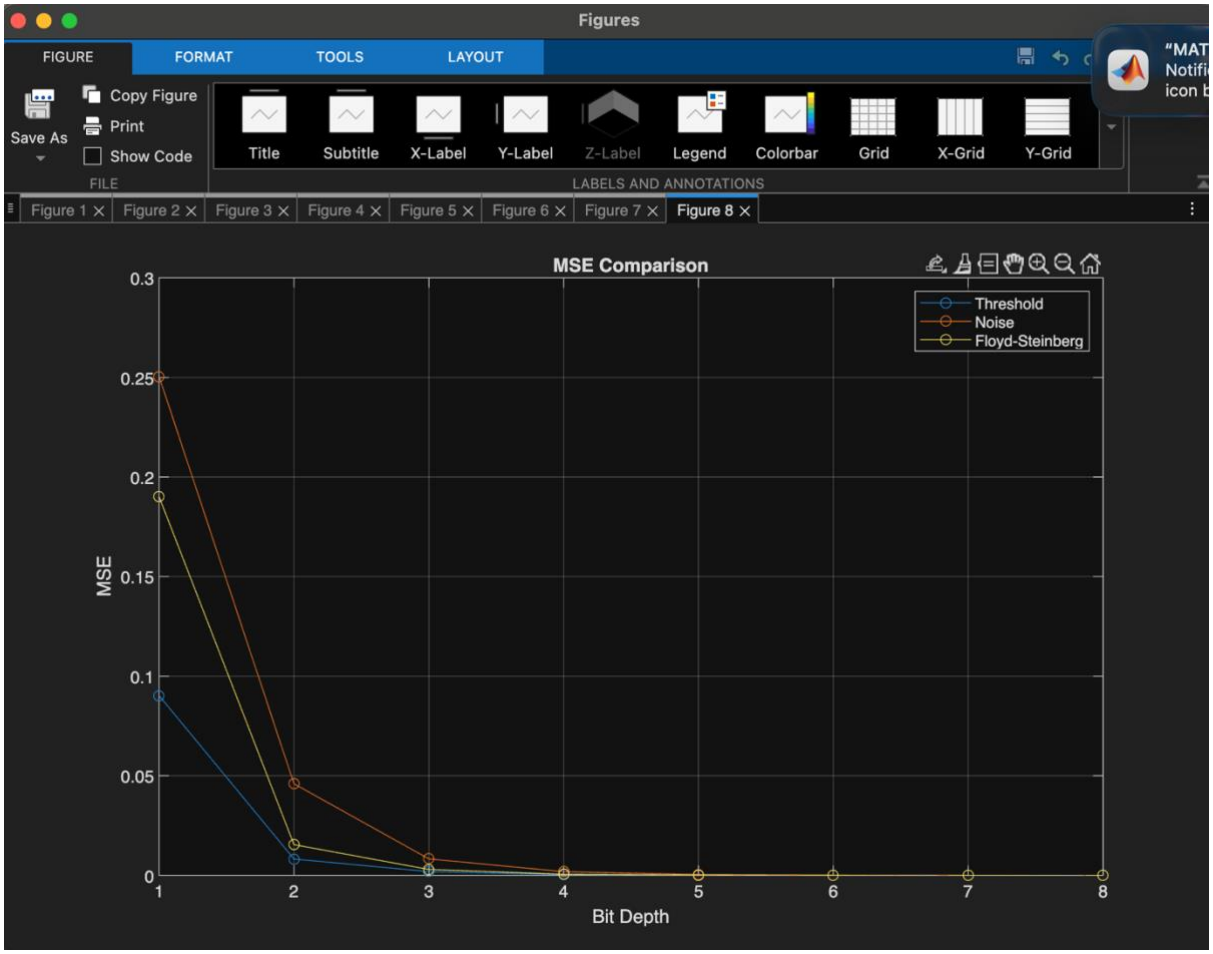
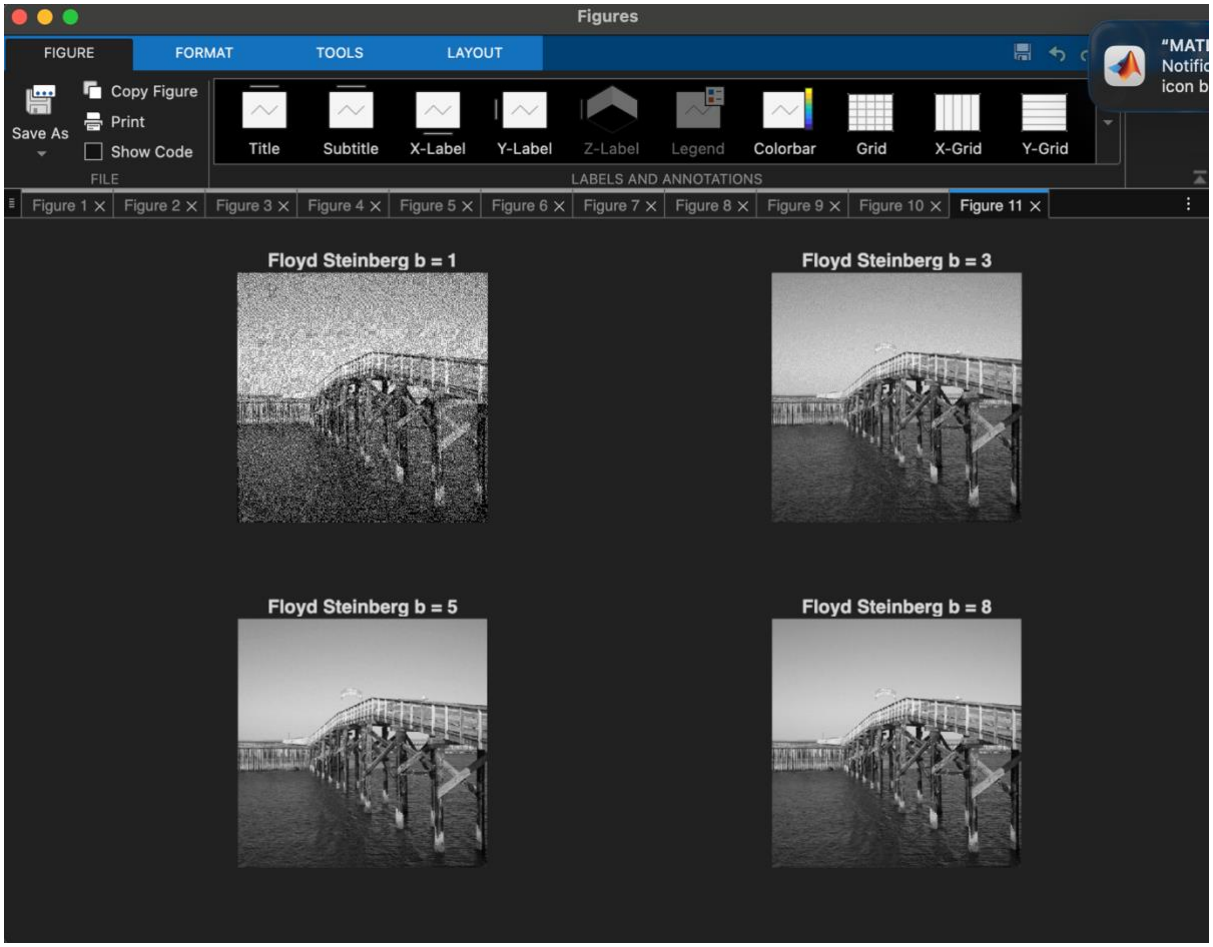
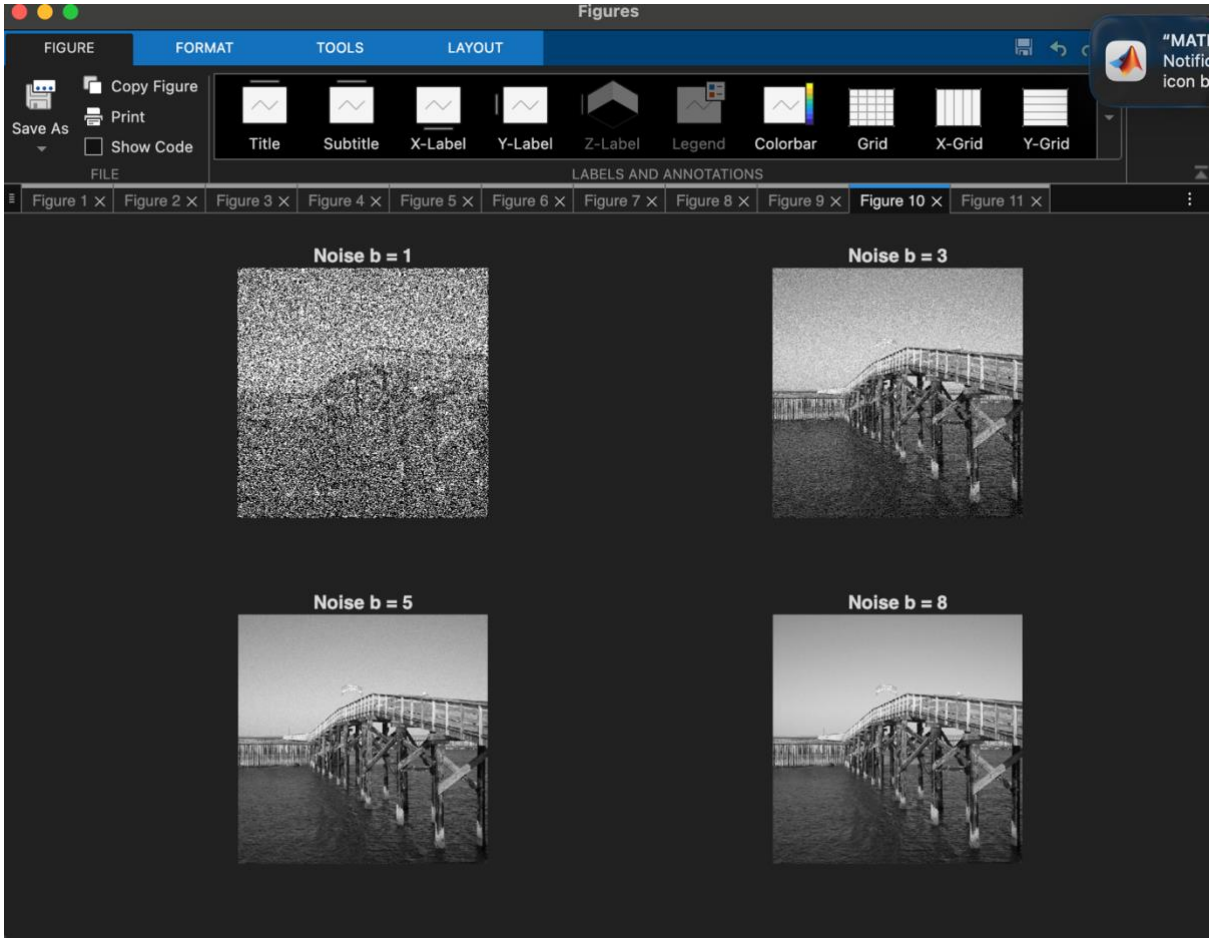
Part 1

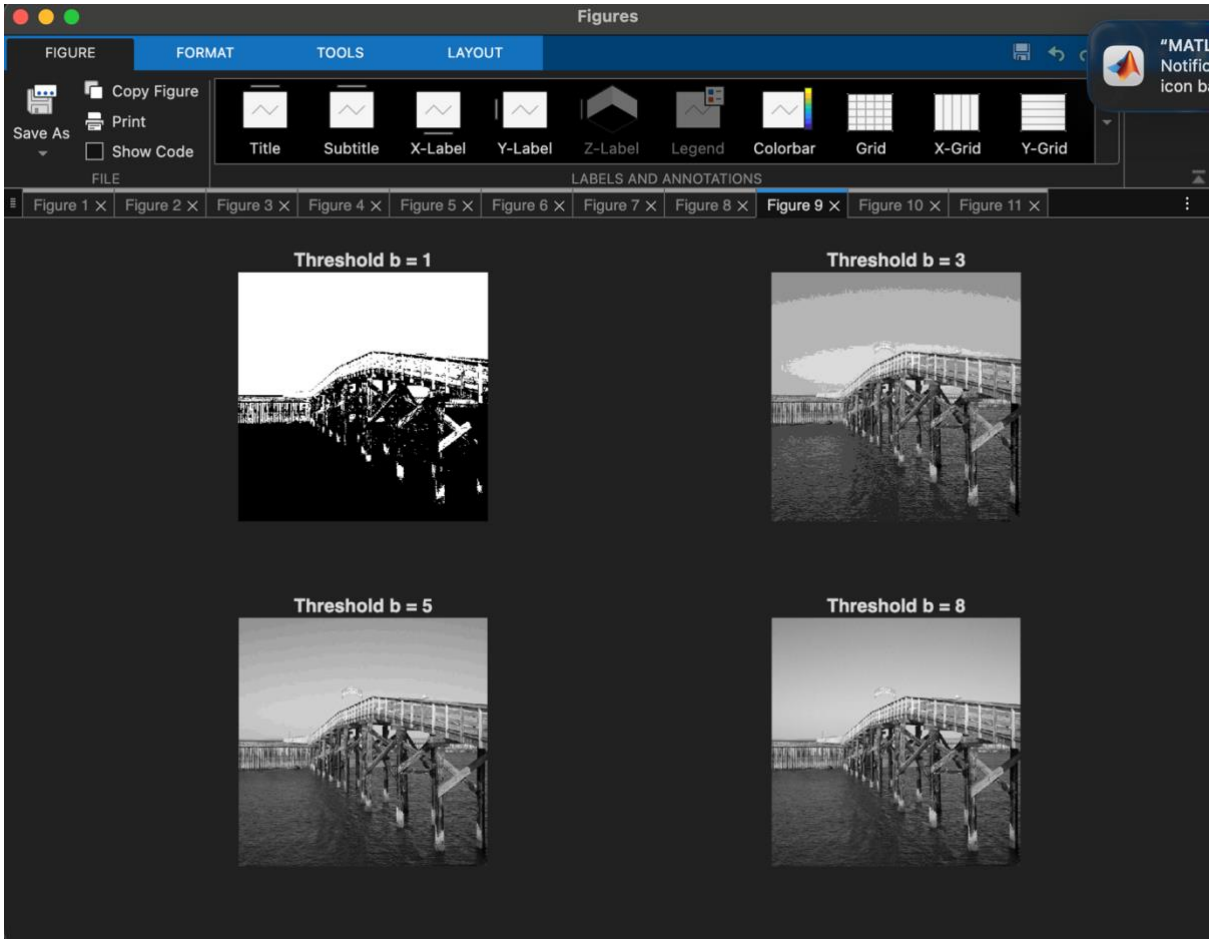

Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 1
Figure 2
Figure 3
Figure 4
Figure 5
Figure 6
Figure 7
Figure 8
SSIM Comparison
Threshold
Noise
Floyd-Steinberg
SSIM
Bit Depth
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
1
2
3
4
5
6
7
8



As we can see in the plots (first 6 plots are MSE, SSIM printed individually. Figure 7 and 8 are comparison plots), MSE drops as big depth increases. This makes sense because the dithered image becomes more and more similar to the original image. SSIM rises because of the same reason.


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 1
Figure 2
Figure 3
Figure 4
Figure 5
Figure 6
Figure 7
Figure 8
Figure 9
Figure 10
Figure 11
Floyd Steinberg b = 1
Floyd Steinberg b = 3
Floyd Steinberg b = 5
Floyd Steinberg b = 8


Figures
FIGURE
FORMAT
TOOLS
LAYOUT
Save As
Copy Figure
Print
Show Code
FILE
Title
Subtitle
X-Label
Y-Label
Z-Label
Legend
Colorbar
Grid
X-Grid
Y-Grid
LABELS AND ANNOTATIONS
Figure 1
Figure 2
Figure 3
Figure 4
Figure 5
Figure 6
Figure 7
Figure 8
Figure 9
Figure 10
Figure 11
Noise b = 1
Noise b = 3
Noise b = 5
Noise b = 8



When b is small, for threshold dithering, I find obvious banding. Noise dithering is barely recognizable, and Floyd Steinberg did the best. This is probably because FS focuses on keeping partial brightness evenness. It spread errors to neighbours.

From the plot, I found Floyd Steinberg's data falls in between threshold and noise, which implies that data does not necessarily equal to observation.

I find that when b is small, both thresholding and FS dithering are recognizable. Thresholding does better in shape while FS does better in texture. When b rises, noise and FS do better than thresholding as there is large chunks of banding in threshold dithering.

When bit depth increases, I noticed there is a diminished return in image quality. This is because error is diminishing, and thus FS does not seem to be outstanding any more.

I used 3 algorithms for 3 dithering methods.

Threshold: I let function to take image and bit depth, calculate how many levels there should be, and round value to the nearest level.

Noise: add a random noise on top of threshold.

FS: I swept pixels from left to right then from top to down (column -> row). For every pixel, round its greyscale and compute error by comparing to the original value. Spread the error to 4 directions using 4 fixed rates.

In main method, I just reused those functions and derived data.

Part 2

1. What compression rate does bzip2 achieve on the file?

Original file: 7.3 MB. After bzip2: 4.9 MB rate: 67%

2. What compression rate does bzip2 achieve on l = 1 file?

Original file: 7.3 MB. After bzip2: 3.5 MB rate: 48%

I used

“maxDifference_l3 = max(abs(int32(signal) - reconstructedSignal_l3));**” to check original signals and reconstructed signals are the same. So there is no information lost**

3. **Original file: 7.3 MB. After bzip2: 3.1 MB rate: 42%**
4. **Original file: 7.3 MB. After bzip2: 3 MB rate: 41%**
   I came up with l = 3 (3, -3, 1) by following the quadratic coefficients.
   (1-z^(-1))^3 gives me 3 coefficients.

Algorithm I used:

to do prediction, I used l = 1, =2, =3 and used the same number of signals and assign them weight to compute error.

Using “prediction = coefficient[1] * signal[n - 1] + coefficient[2] * signal[n - 2] + ... + coefficient[l] * si signal[n - l]**”, I was able to compute an error list. And later I followed the same pattern for recovery (reconstruction)**